Projeto da Disciplina Sistemas Distribuídos

Amanda Almeida Cardoso

Samuel Magalhães Pereira

Ponto a Ponto - Uma implementação do jogo Dots and Boxes

Histórico de Revisões do Documento

Revisão	Descrição	Modificado por	Status	Data
1.0	**Descrição do Projeto** 1.1 Escopo 1.2 Requisitos 1.3 Tecnologias	Amanda Cardoso Samuel Pereira	Concluído	14/10/2025
2.0	**Arquitetura Geral da Solução - Diagrama de Componentes** 2.1 Cliente-Servidor com WebSocket 2.2 Diagrama de Componentes 2.3 Funcionalidades Iniciais	Amanda Cardoso Samuel Pereira	Concluído	25/11/2025
3.0	**Modelo de Dados** 3.1. Esboço do Modelo de Dados 3.2. Modelo de Dados no Cliente 3.3. Modelo de Dados no Servidor **Camada de Rede e Comunicação** 4.1. Autoloads 4.2. Camada de Rede (network.gd) 4.3. Uso de RPCs 4.4. Estrutura dos payloads (messages.gd) **Diagramas** 5.1. Diagrama de Classes 5.2. Diagrama de Estados **Telas de Jogo**	Amanda Cardoso Samuel Pereira	Concluído	15/01/2026

1 Descrição do Projeto

1.1 Escopo

Este projeto tem como objetivo a implementação do jogo Dots and Boxes (também conhecido como “Timbriche” no Brasil), tradicionalmente jogado em papel e caneta. Criado em 1889 por Édouard Lucas, o jogo é baseado em uma malha de pontos conectáveis, na qual os jogadores alternam turnos marcando arestas com o intuito de formar quadrados de 1x1.

Cada vez que um jogador completa um quadrado, ele ganha um ponto e tem direito a jogar novamente. O jogo termina quando toda a malha é completada, e o vencedor é aquele que possuir o maior número de quadrados conquistados.

A implementação tem como objetivo reproduzir as regras originais, oferecendo aos usuários, mesmo que geograficamente distantes, uma experiência equivalente à de jogar o jogo em sua forma tradicional, no papel.

O desenvolvimento será feito através da Godot, uma game engine que possui ferramentas diversas tais como: multiplataforma, networking utilizando protocolos UDP e TCP, e editor de interfaces gráficas. Serão adotadas boas práticas de desenvolvimento, como programação orientada a objetos, clareza de código e controle de versão com Git.

Link para o Repositório (branch master): https://github.com/samuelsmp03/Trabalho-SD

1.2 Requisitos

Todos os requisitos foram elaborados a partir de uma análise das regras do jogo e das principais interações do jogador com o sistema. Os Requisitos Funcionais estão organizados de acordo com uma visão geral do funcionamento do jogo, o que facilita a compreensão do comportamento esperado em cada etapa da partida.

A estrutura de tabelas abaixo está dividida nas seguintes seções:

- Criação do Jogo e Estrutura do Tabuleiro
- Tela de Espera
- Participação do Jogador
- Requisitos Não Funcionais
- Regras de Negócio

1.2.1 Requisitos Funcionais

Criação do Jogo e Estrutura do Tabuleiro

ID	Nome	Elemento do jogo	Descrição
RF001	Criação de Tabuleiro	Tabuleiro	Ao iniciar a partida, o sistema deve criar automaticamente um tabuleiro de dimensão 5x5 composto por uma malha de ponto conectáveis por arestas.
RF002	Acesso à tela inicial	Interface	O sistema deve permitir ao jogador criar um novo jogo ou escolher um existente na tela inicial.
RF003	Definição de Configurações Iniciais	Criação da Partida	O sistema deve permitir ao criador do jogo definir o número de jogadores (mínimo 2, máximo 5), seu nome de usuário e cor de preferência.
RF004	Exibição de Partidas Criadas	Interface	O sistema deve tornar o jogo criado visível na tela inicial, permitindo que outros jogadores entrem.
RF005	Geração de Link de Convite	Sala de Jogo	O sistema deve permitir exportar um link de convite para que outros usuários ingressem na sala.

Tela de Espera

ID	Nome	Elemento do jogo	Descrição
RF006	Exibição da Tela de Espera	Interface	Ao clicar em um jogo, o sistema deve exibir uma tela de espera com o número atual e o máximo de jogadores, além de um cronômetro regressivo de 60 segundos.
RF007	Início Automático da Partida	Sala de Espera	O sistema deve iniciar automaticamente o jogo quando o número de jogadores atingir o valor definido na criação
RF008	Encerramento da Sala de Espera	Sala de Espera	O sistema deve encerrar automaticamente a sala de espera quando o tempo limite for atingido sem o número mínimo de jogadores.

Participação do Jogador

ID	Nome	Elemento do jogo	Descrição
RF009	Escolha de Identificação	Jogador	O sistema deve permitir ao jogador escolher seu nome de usuário e uma cor para representá-lo no tabuleiro.
RF010	Ordem de Turnos	Mecânica	O jogo deve iniciar com o criador tendo o primeiro turno, e os demais jogadores devem ter suas ordens sorteadas aleatoriamente.
RF011	Seleção de Aresta	Tabuleiro	O sistema deve permitir a seleção de uma aresta válida entre dois pontos. Ao passar o mouse sobre a aresta, ela deve mudar de cor para indicar que é selecionável.

RF012	Marcação de Quadrado Completo	Mecânica	Quando uma jogada completar um quadrado, o sistema deve marcá-lo com a cor e a inicial do usuário do jogador, atribuir um ponto e permitir que ele jogue novamente.
RF013	Encerramento Automático	Mecânica	O sistema deve encerrar automaticamente o jogo quando todos os quadrados estiverem preenchidos.
RF014	Exibição do Ranking Final	Interface	Ao término da partida, o sistema deve exibir um ranking com o nome dos jogadores e a quantidade de quadrados completados.
RF015	Gerenciamento de Cores	Jogador	O sistema deve gerenciar as cores em tempo real; caso a cor escolhida esteja indisponível, uma cor alternativa deve ser atribuída automaticamente.

1.2.2 Requisitos Não Funcionais

ID	Nome	Elemento do jogo	Descrição
RNF01	Capacidade de Jogadores	Escalabilidade	O sistema deve gerenciar até 5 jogadores simultâneos por sala.
RNF02	Reconexão Automática	Conectividade	Em caso de desconexão, o sistema deve tentar reconexão por até 10 segundos
RNF03	Encerramento da Sala de Espera	Rede	O sistema deve garantir consistência total entre todos os jogadores (estado idêntico do tabuleiro e pontuação)

RNF04	Fácil visualização	Interface	A interface deve permitir fácil identificação das jogadas e dos jogadores
RNF05	Controle de Turnos	Mecânica	O sistema deve impedir jogadas fora do turno do jogador
RNF06	Validade dos Convites	Sala de Jogo	Os links de convite devem ser únicos e válidos apenas para a sala correspondente

1.2.3. Regras de Negócio

ID	Nome	Elemento do jogo	Descrição
RN1	Pontuação e Turno Extra	Mecânica	O jogador que completa um quadrado recebe 1 ponto e pode jogar novamente
RN2	Identificação Visual	Jogador	As cores dos jogadores devem ser visualmente distintas para evitar confusões.
RN3	Plataforma de Execução	Sistema	O jogo deve ser executado em um navegador web.
RN4	Tempo de Jogada	Mecânica	O tempo limite de cada jogada é de 10 segundos; se excedido, a vez passa automaticamente ao próximo jogador

1.3 Tecnologias

Godot

Game engine de código aberto e multiplataforma, que se destaca por sua arquitetura modular, permitindo uma boa organização de projeto. Para o desenvolvimento do jogo, destacam-se as características e funcionalidades.

Em relação à transmissão de dados, a engine oferece uma API integrada que suporta o protocolo TCP, que foi selecionado para garantir a entrega ordenada e confiável dos dados. Já para a interface gráfica, a godot possui um sistema completo dedicado à criação de interfaces de maneira visual. Permitindo assim construir de forma intuitiva todos os elementos gráficos do jogo.

Para o suporte à web, a Godot permite a utilização de tecnologias de desenvolvimento de jogos na web. Por exemplo, a WebGL utilizada para a renderização de gráficos 2D e o WebAssembly que permite a execução do código do jogo com performance próxima à nativa, garantindo uma experiência fluida para o jogador. Sobre a linguagem de programação em si, a plataforma oferece diversas opções para a implementação da lógica do jogo. Com a possibilidade de utilização de sua linguagem própria, o GDScript, ou com o C# com o framework .NET, ou ainda C++ através do sistema GDExtension.

2 Arquitetura Geral da Solução e Diagrama de Componentes

Antes de definir a arquitetura, é necessário refletir sobre a natureza do jogo e sobre as necessidades do projeto. Trata-se de um jogo multiplayer para navegador, baseado em turnos, no qual cada jogador se conecta remotamente a partir de sua própria casa. Como esses jogadores podem estar geograficamente distantes e não necessariamente na mesma rede local, precisamos de um mecanismo de comunicação capaz de conectar clientes distribuídos por meio do navegador. Nesse contexto, faz sentido delegar parte do processamento ao cliente, como a validação da própria jogada e a manutenção de um estado local do tabuleiro, reduzindo a carga sobre o servidor e evitando verificações constantes.

No entanto, o uso do navegador impõe uma limitação arquitetural importante: devido ao sandboxing, navegadores não podem aceitar conexões TCP ou UDP diretamente, o que inviabiliza conexões diretas entre os clientes. Isso significa que qualquer comunicação entre jogadores precisa obrigatoriamente passar por um servidor intermediário usando WebSockets, já que o export HTML5 do Godot oferece apenas WebSocket e WebRTC como tecnologias de rede. Com essa restrição em mente, ao analisarmos as opções disponíveis na plataforma Godot, a escolha da arquitetura torna-se clara: utilizar uma arquitetura Cliente-Servidor com WebSocket.

2.1. API MultiPlayer do Godot

O Godot fornece uma API Multiplayer de alto nível baseada em SceneTrees, que lida automaticamente com aspectos como chamadas RPC, sincronização, identificação de peers e autoridade do servidor. Além disso, disponibiliza diferentes formas de transporte de rede, como WebSocketMultiplayerPeer, EnetMultiplayerPeer e WebRTCMultiplayerPeer. O fluxo geral funciona da seguinte forma: primeiro inicializa-se o peer de transporte e, em seguida, esse peer é entregue à MultiplayerAPI, que passa a gerenciar a comunicação de alto nível no SceneTree.

É importante notar que essa API de alto nível não resolve toda a lógica de jogo. Por exemplo, ela não fornece mecanismos de coordenação específicos, como a passagem de tokens entre jogadores; isso deve ser implementado pelo desenvolvedor.

Sendo assim, para escolher o transporte adequado, é preciso entender as três opções que o Godot oferece e explicar por que usar a escolhida e por que não usar as outras. Como mencionado, existem três implementações principais: WebSocketMultiplayerPeer, WebRTCMultiplayerPeer e EnetMultiplayerPeer. Todas elas se apoiam em um objeto intermediário chamado MultiplayerPeer, que estende PacketPeer e herda métodos de serialização, envio e recebimento de dados. Esse objeto também gerencia múltiplos peers remotos, atribui IDs, detecta conexões e desconexões e define modos de transferência, servindo como uma camada de abstração comum para diferentes tecnologias de rede. Embora seja possível instanciar MultiplayerPeer diretamente, isso é desaconselhado, uma

vez que a prática correta é utilizar as implementações concretas fornecidas pelo Godot, como WebSocketMultiplayerPeer, ENetMultiplayerPeer ou WebRTCMultiplayerPeer.

A primeira opção, o EnetMultiplayerPeer, utiliza a biblioteca ENet, projetada para jogos que necessitam de comunicação UDP de baixa latência, com confiabilidade e ordenação de pacotes. Embora adequada à vários contextos, essa escolha não se encaixa na essência do jogo Ponto a Ponto: não há necessidade de transmissão frequente de dados em alta velocidade, e, mais importante, ENet não pode ser utilizado no navegador, pois este não permite conexões UDP diretas. Portanto, seu uso não faz sentido para este trabalho.

Outra alternativa é o WebRTCMultiplayerPeer, que implementa comunicação rápida, segura e em tempo real entre dois pares, essencialmente uma forma de arquitetura P2P. Para estabelecer conexões WebRTC, é necessário um servidor de sinalização. A comunicação começa com os pares conectando-se a esse servidor e enviando suas informações para ele. O servidor, então, repassa esses dados aos demais pares, permitindo que eles estabeleçam a conexão direta desejada. Uma vez conectados entre si, os pares podem se desconectar do servidor de sinalização e manter suas conexões P2P abertas.

O uso do WebRTC seria possível e até interessante, mas jogos que adotam P2P geralmente lidam com cenários em que os peers precisam manter um estado compartilhado que evolui de forma contínua e que deve permanecer idêntico entre todos a cada instante. Esse tipo de arquitetura demanda mecanismos de consenso e resolução de conflitos para garantir que o estado global permaneça consistente entre os pares. No entanto, esse modelo não se aplica a um jogo baseado em turnos, onde não há simulação contínua nem necessidade de que todos os peers concordem a cada instante sobre o estado atualizado. Além disso, no contexto de um jogo em navegador, ainda precisaríamos de WebSocket para o lobby e a conexão inicial, o que adicionaria complexidade extra sem necessidade, já que o próprio WebSocket oferece tudo o que precisamos para o jogo. Por esses motivos, apesar de ser tecnicamente viável, o WebRTC não se alinha bem à natureza do jogo e foi descartado.

Por fim, escolhemos o WebSocketMultiplayerPeer. Esta implementação permite criar clientes e servidores via WebSocket, funcionando perfeitamente no navegador e integrando-se naturalmente à MultiplayerAPI.

2.2. Cliente-Servidor com WebSocket

Em termos gerais, o sistema adotará a arquitetura Cliente-Servidor em conjunto com WebSockets, permitindo a troca contínua de mensagens entre os componentes. A arquitetura Cliente-Servidor fundamenta-se na interação entre cliente e servidor, onde o cliente envia requisições e o servidor as processa e responde (TANENBAUM, 2023). Apesar desse modelo clássico se basear em requisições pontuais, neste trabalho utilizaremos uma variação baseada em WebSockets, que permite comunicação bidirecional contínua. No nosso caso, o servidor não validará todas as regras de jogo: o cliente mantém a lógica local, valida suas próprias ações e as envia ao servidor, que fica responsável por gerenciar

partidas, conexões, desconexões, encaminhamento de mensagens, entre outras funcionalidades. Ainda assim, a distinção clara entre cliente e servidor torna essa arquitetura adequada ao projeto.

Como a comunicação tradicional baseada em requisições não atenderia às necessidades de um jogo multiplayer, adotamos WebSockets (FETTE; MELNIKOV, 2011), um protocolo que possibilita comunicação bidirecional e persistente por meio de uma única conexão TCP entre navegador e servidor. A API de WebSocket fornece as interfaces necessárias para manter uma conexão ativa durante toda a partida, permitindo o fluxo de dados em tempo real. Essas características tornam o uso de WebSockets vantajoso no contexto do jogo, já que cada jogador permanecerá conectado e poderá transmitir suas jogadas e atualizações de estado de forma eficiente.

2.3. Papel do Servidor X Papel do Cliente

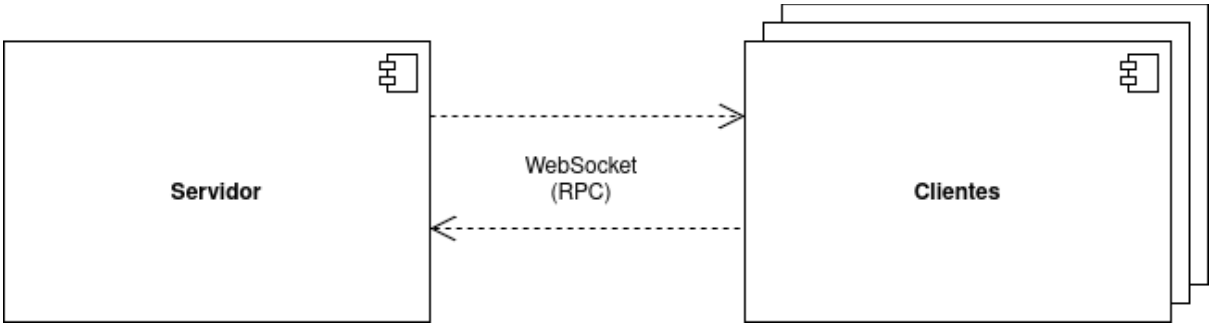
No modelo adotado, o servidor exerce um papel central de coordenação, enquanto os clientes são responsáveis pela execução da lógica de jogo. O servidor mantém as conexões ativas, gerencia a criação e encerramento de partidas, controla a entrada e saída de jogadores e encaminha mensagens entre os participantes. Além disso, ele garante a organização interna de cada sala de jogo, incluindo a ordem de turno e o repasse das jogadas recebidas aos demais jogadores.

Os clientes, por sua vez, mantêm a lógica do jogo localmente. Cada jogador é responsável por validar sua própria jogada, atualizar seu estado local e enviar sua ação ao servidor por meio de WebSockets. Após receber as atualizações encaminhadas pelo servidor, cada cliente sincroniza seu estado local com as jogadas dos demais participantes. Dessa forma, apesar de o servidor atuar como coordenador da partida, a lógica e a validação do jogo permanecem distribuídas nos clientes, reduzindo a carga do servidor.

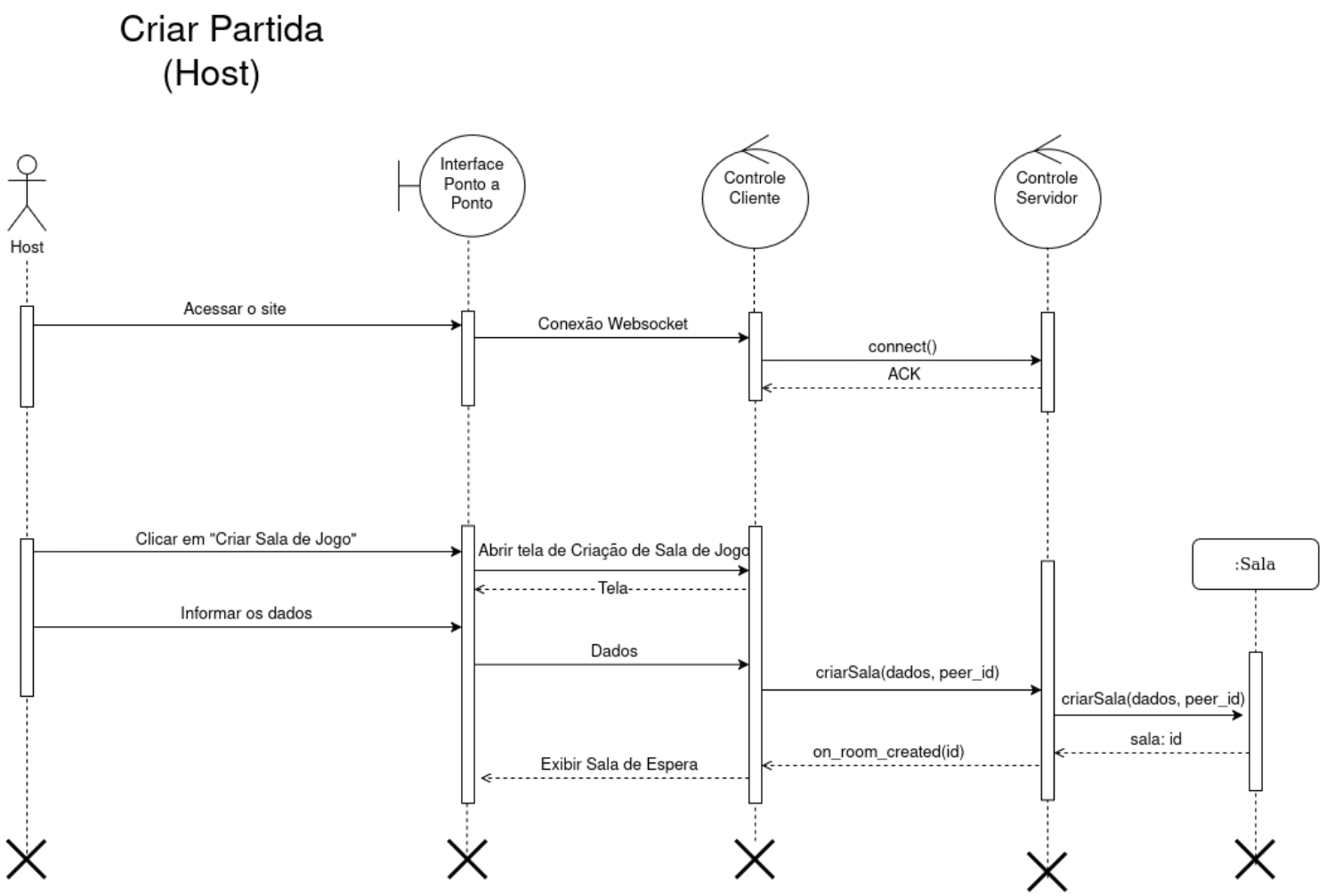
Vale ressaltar que o papel de identificar o término da partida será atribuído ao cliente. Quando o cliente detecta o fim do jogo em seu estado local, ele notifica o servidor, que repassa essa informação aos demais jogadores. Para facilitar a visualização, os papéis de cada um estão dispostos na tabela abaixo:

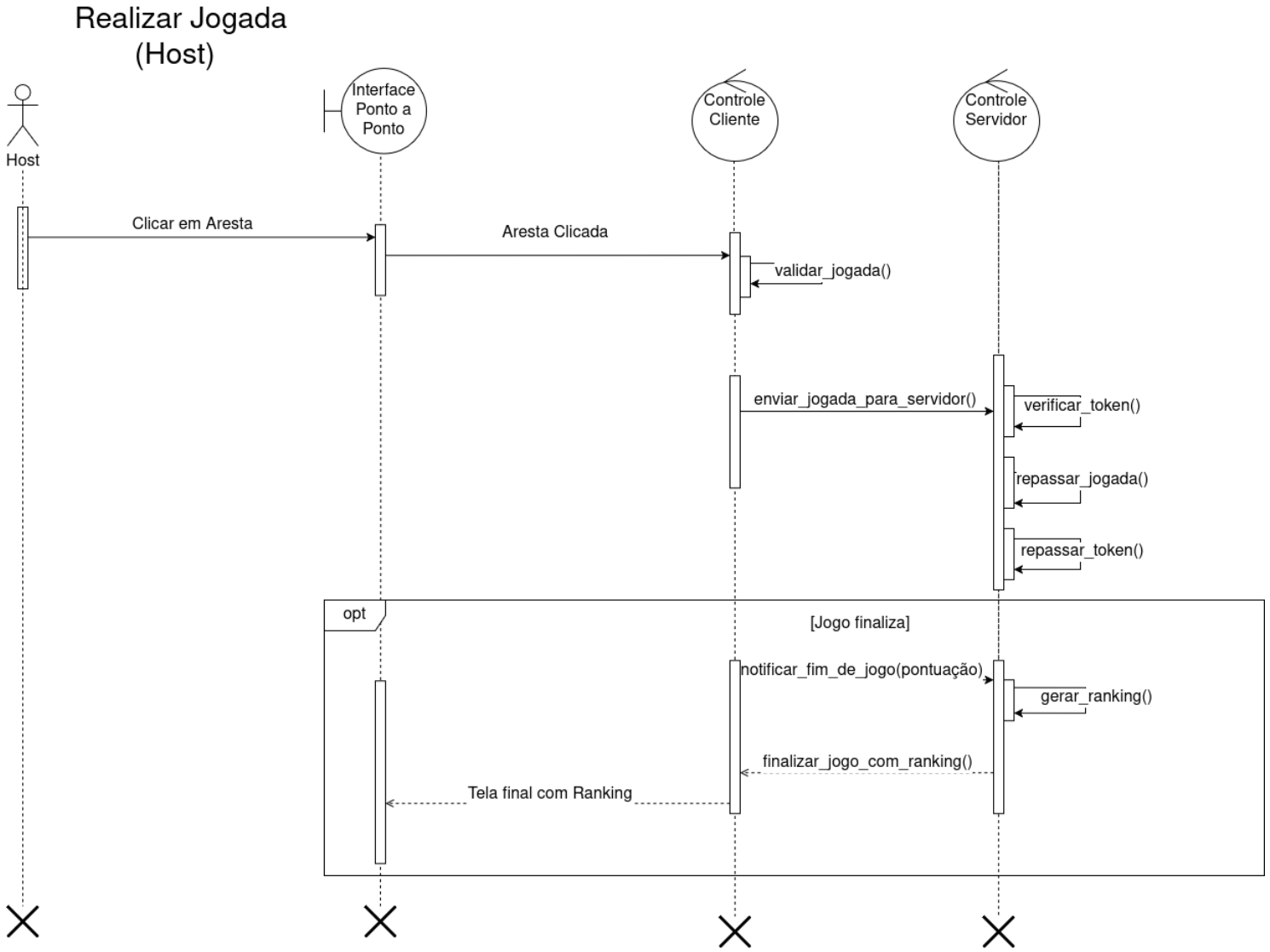
Servidor	Cliente
Gerencia conexões e desconexões	Permite interação do usuário
Cria salas	Mantém o estado local do tabuleiro
Controla entrada e saída de jogadores	Valida a própria jogada antes de enviá-la
Encaminha mensagens entre jogadores	Envia jogadas ao servidor por WebSocket
Organiza turnos e estrutura lógica dos jogadores na sala	Atualiza seu estado ao receber mensagens do servidor
Recebe mensagem de término da partida e envia notificações	Envia sinal ao servidor informando de térmido de partida

2.4. Diagrama de Componentes

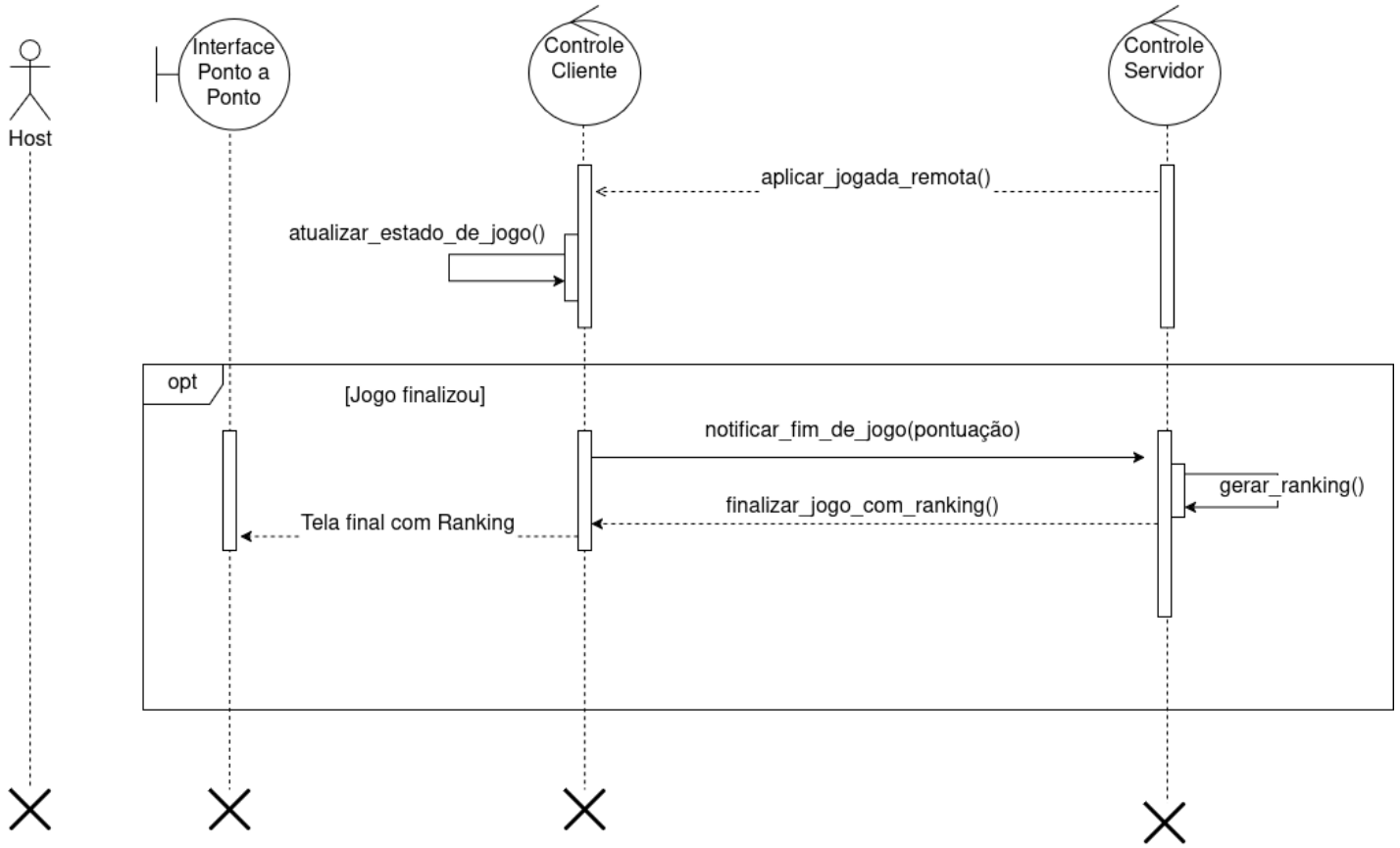


2.5. Diagramas de Sequência




Realizar Jogada
(Host)
Host
Interface Ponto a Ponto
Controle Cliente
Controle Servidor
Clicar em Aresta
Aresta Clicada
validar_jogada()
enviar_jogada_para_servidor()
verificar_token()
repassar_jogada()
repassar_token()
opt
[Jogo finaliza]
notificar_fim_de_jogo(pontuação)
gerar_ranking()
finalizar_jogo_com_ranking()
Tela final com Ranking

Receber Jogada (HOST)



3. Modelo de Dados

3.1. Esboço do Modelo de Dados

O sistema adota um modelo cliente–servidor com autoridade central no servidor. Os dados são organizados em dois contextos distintos: dados mantidos no servidor e dados mantidos no cliente.

O servidor mantém o estado da sessão e da partida, incluindo salas existentes, jogadores conectados, status da sala e controle de turno. O cliente mantém dados do jogador local e o estado local do tabuleiro, sendo responsável pela interação com a interface gráfica, pela aplicação visual das jogadas recebidas e pela validação preliminar das próprias jogadas antes do envio ao servidor.

3.2. Modelo de Dados no Cliente

Estado global do cliente (*global_data.gd*)
O cliente mantém um estado global compartilhado entre cenas. Esse estado concentra: dados do jogador local, dados da sala (incluindo turno e status), configurações da partida e variáveis temporárias usadas no fluxo de criação/entrada em sala. O mesmo script também centraliza eventos de UI por meio de uma fila e do sinal *ui_event_pushed*.

Estado local do tabuleiro (*board_state*)
O tabuleiro é composto por linhas horizontais e verticais que podem ser traçadas pelos jogadores, formando caixas ao longo da partida. O estado do tabuleiro é descrito por variáveis que registram quais linhas já foram desenhadas, quais caixas foram completadas e as dimensões da malha. A partir dessas informações, o cliente consegue identificar o fechamento de caixas, atualizar a pontuação visualmente e detectar o término da partida.

3.3. Modelo de Dados no Servidor

No servidor, os dados são organizados em estruturas que representam o estado da sessão dos jogadores e o estado das salas. Além das classes *PlayerState* e *RoomState*, o servidor utiliza dicionários para organização e acesso eficiente ao estado.

PlayerState no Servidor: Representa a sessão de um jogador conectado. Armazena apenas informações necessárias para identificação e associação com uma sala:

```
{
  "peer_id": int,
  "name": String,
  "room_id": String,
  "color": String,
  "connected": bool
}
```

RoomState no Servidor: Representa o estado de uma sala mantido pelo servidor.

Armazena os jogadores participantes, o jogador associado ao papel de host, o jogador que possui o turno, o status da sala e as configurações da partida:

```
{
  "room_id": String,
  "players": Array[int],
  "host": int,
  "token_owner": int,
  "status": String,
  "target_player_count": int,
  "board_size": int
}
```

O servidor utiliza essas estruturas para validar requisições recebidas dos clientes (como existência da sala, associação do jogador à sala, entre outros), controlar o turno e enviar atualizações de estado para os participantes da sala. O host, nesse caso, é utilizado para guardar a pessoa que criou a sala, de forma que isso possa ser exibido na UI na lista de salas disponíveis.

4. Camada de Rede e Comunicação

A comunicação do sistema foi estruturada a partir dos mecanismos de multiplayer do Godot, combinados com abstrações próprias do projeto. Para isso, são utilizados autoloads e um script dedicado à rede, responsável por centralizar as chamadas RPC e desacoplar a lógica de comunicação da lógica de jogo.

4.1. Autoloads

Autoloads são scripts carregados automaticamente pelo Godot no início da execução da aplicação e mantidos durante todo o ciclo de vida do jogo. Do ponto de vista funcional, se comportam como *singletons*, ficando disponíveis globalmente independentemente da cena ativa. O uso de autoloads permite que cenas acessem funcionalidades centrais sem conhecer detalhes de implementação de outras partes do sistema. No projeto, os autoloads são utilizados para centralizar responsabilidades que não pertencem a uma cena específica. Os principais autoloads utilizados:

Global (*global_data.gd*): mantém o estado global do cliente, como dados do jogador, dados da sala, configurações da partida e eventos de UI.

NetworkManager (*network.gd*): centraliza toda a comunicação de rede entre cliente e servidor.

ClientLogic(client_logic.gd): permite acesso global às funcionalidades do cliente, funcionando como uma camada intermediária entre as cenas e o NetworkManager.

4.2. Camada de Rede (*network.gd*)

A comunicação entre cliente e servidor é centralizada no script *network.gd*, que atua como uma camada intermediária entre o cliente e o servidor. Essa camada funciona como uma ponte, com as seguintes responsabilidades:

- Encapsular chamadas RPC.
- Padronizar o envio e o recebimento de payloads.
- Emitir sinais locais a partir de eventos de rede.

Nenhuma cena ou lógica de jogo realiza chamadas RPC diretamente. Em vez disso, elas interagem com o *NetworkManager*, que encaminha as requisições ao servidor ou aos clientes apropriados.

4.3. Uso de RPCs

A comunicação entre cliente-servidor é realizada por meio de chamadas RPC utilizando o sistema de multiplayer do Godot. Cada RPC transporta um payload estruturado, contendo apenas os dados necessários para a execução da ação.

No fluxo cliente–servidor:

- O cliente envia requisições RPC ao servidor para ações como criação de sala, entrada em sala e envio de jogadas.
- O servidor processa a requisição, valida e decide se a ação é aplicada.
- Após o processamento, o servidor utiliza RPCs para propagar eventos autoritativos aos clientes, como início de partida, jogadas válidas e encerramento do jogo.

No *NetworkManager*, cada RPC recebida é convertida em um sinal local, permitindo que o *ClientLogic* ou *ServerLogic* reajam ao evento.

4.4. Estrutura dos payloads (*messages.gd*)

Os payloads utilizados nas chamadas RPC são centralizados em *messages.gd*. Esse script apenas padroniza a estrutura dos dicionários enviados entre cliente e servidor. Os payloads são montados no *Network.gd*

Função que a utiliza	Direção	Dicionário
create_room	Cliente → Servidor	{"room_id": String, "player_name": String, "player_color": String, "num_players": int, "board_size": int }
join_room	Cliente → Servidor	{ "room_id": String, "player_name": String, "player_color": String }
make_move	Cliente → Servidor	{"move_type": String, "pos_x": int, "pos_y": int, "scored": bool }
request_game_over	Cliente → Servidor	{"ranking": Array, "winner_id": int }.

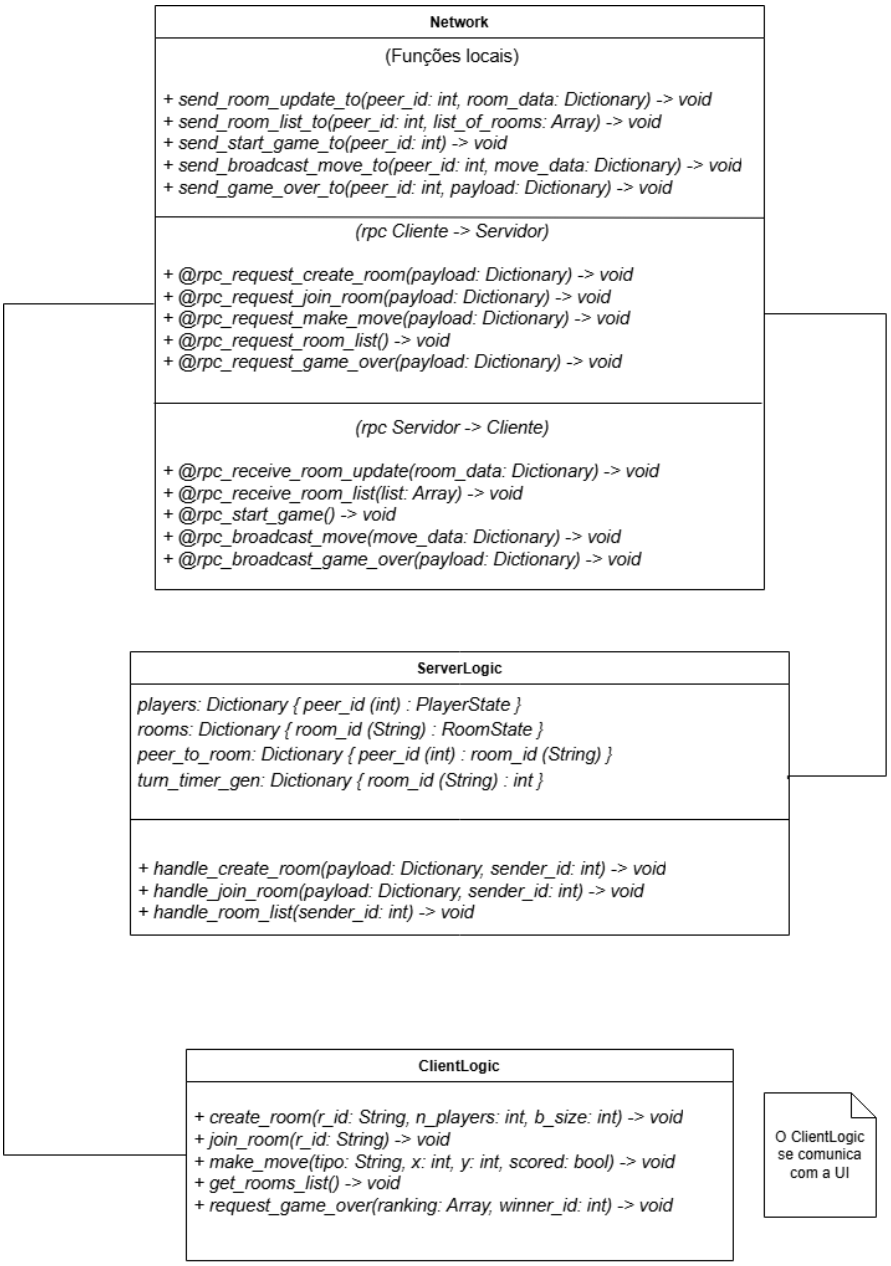
Além desses payloads de requisição, o servidor também envia dicionários de estado montados a partir das estruturas autoritativas (por exemplo, *RoomState.to_dict()* e a lista *players_details*). Esses dados são propagados aos clientes por RPCs de atualização, como *rpc_receive_room_update* e *rpc_receive_room_list*. Abaixo temos a tabela das funções @rpc existentes no *Network.gd.*

Direção	@RPC	Descrição
Cliente → Servidor	*rpc_request_create_room*	Recebe do cliente a solicitação de criação de sala e repassa para ServerLogic.handle_create_room, usando o sender_id para identificar quem pediu.
Cliente → Servidor	*rpc_request_join_room*	Recebe do cliente a solicitação para entrar em uma sala e repassa para ServerLogic.handle_join_room.
Cliente → Servidor	*rpc_request_make_move*	Recebe do cliente uma jogada e repassa para ServerLogic.handle_make_move.
Cliente → Servidor	*rpc_request_room_list*	Recebe do cliente a solicitação de lista de salas (sem payload) e repassa para ServerLogic.handle_room_list.
Cliente → Servidor	*rpc_request_game_over*	Recebe do cliente a solicitação/anúncio de fim de jogo (ranking e winner no payload) e repassa para ServerLogic.handle_game_over.
Servidor → Cliente	*rpc_receive_room_update*	Envia para o cliente uma atualização completa do estado da sala; salva em last_room_data, atualiza Global.room_id e emite o sinal room_updated.
Servidor → Cliente	*rpc_receive_room_list*	Envia para o cliente a lista de salas disponíveis e emite o sinal room_list_updated.
Servidor → Cliente	*rpc_start_game*	Notifica o cliente que o jogo começou e emite o sinal game_started.
Servidor → Cliente	*rpc_broadcast_move*	Faz broadcast (do servidor para clientes) de uma jogada recebida e emite o sinal move_received.
Servidor → Cliente	*rpc_broadcast_game_over*	Faz broadcast (do servidor para clientes) do fim de jogo e emite o sinal game_over.

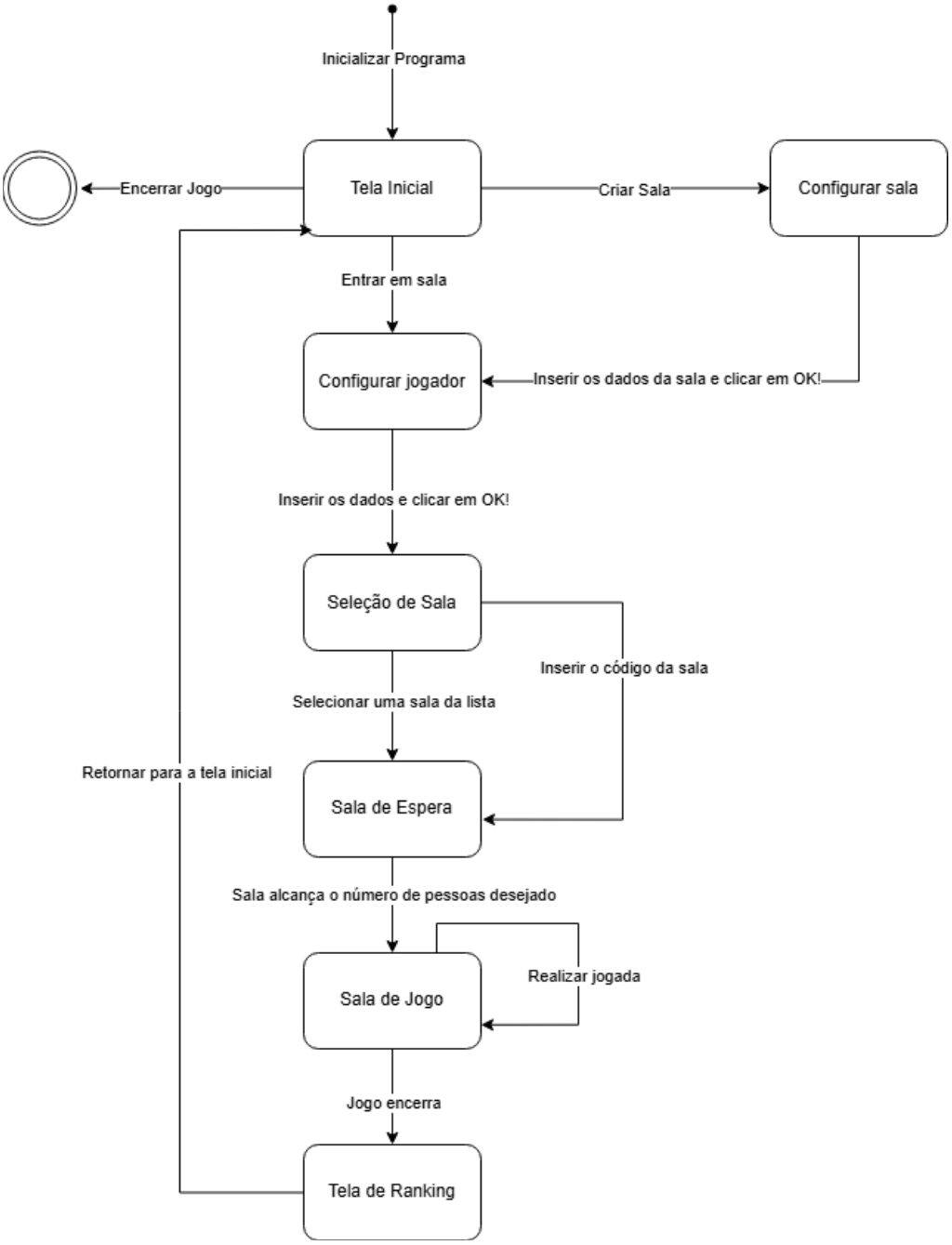
5. Diagramas

5.1 Diagrama de Classes

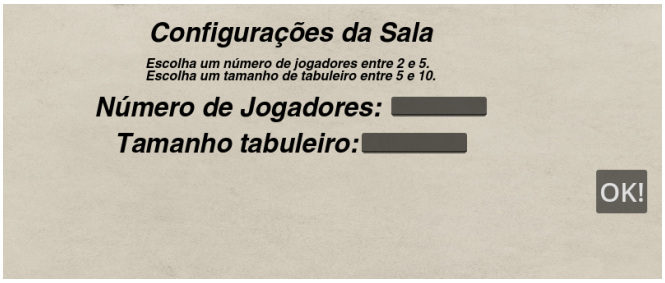
Diagrama de Classes que modela os principais scripts do jogo. Constam as principais funções que fazem o jogo funcionar da maneira adequada. Alguns scripts foram abstraídos para melhor visualização.



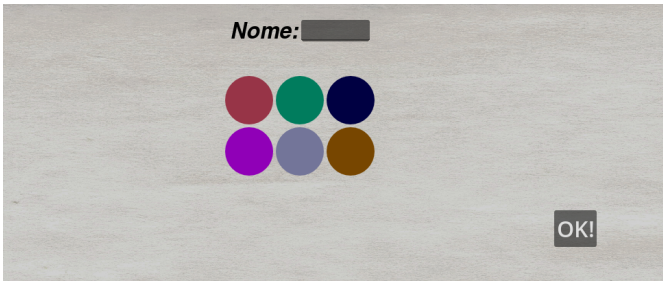
5.2. Diagrama de Estados



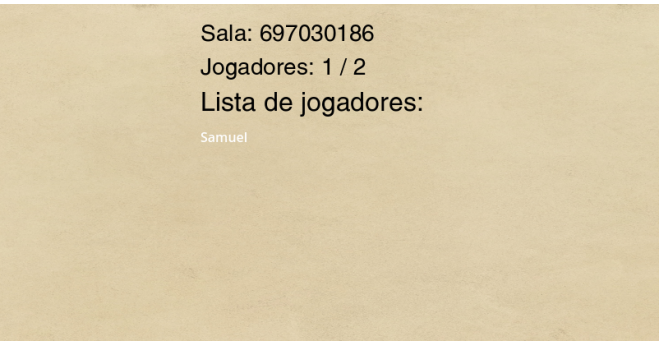
6. Telas de Jogo



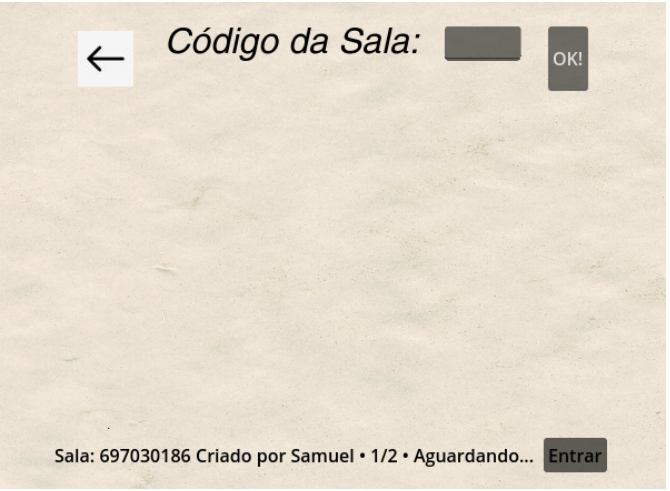
Configurar Sala



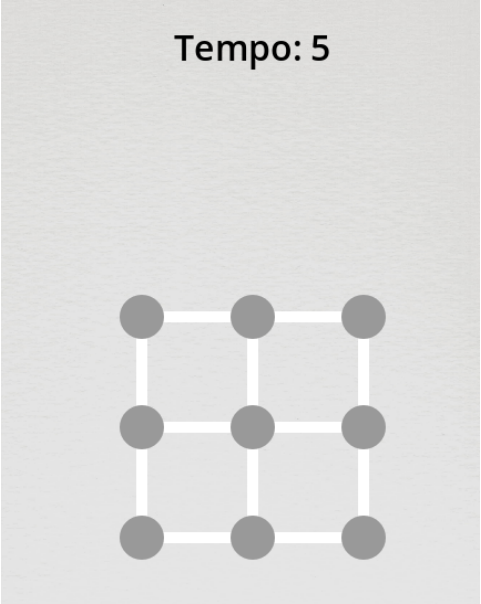
Configurar jogador



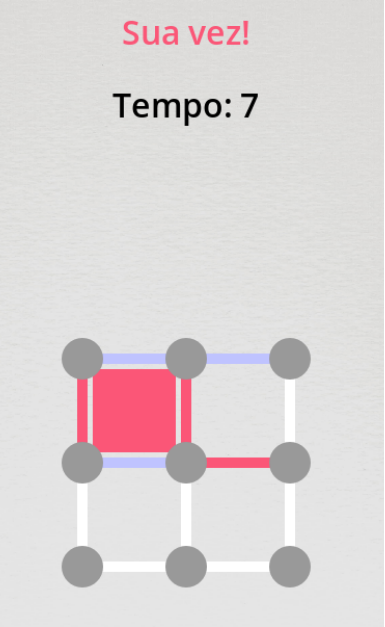
Sala de Espera



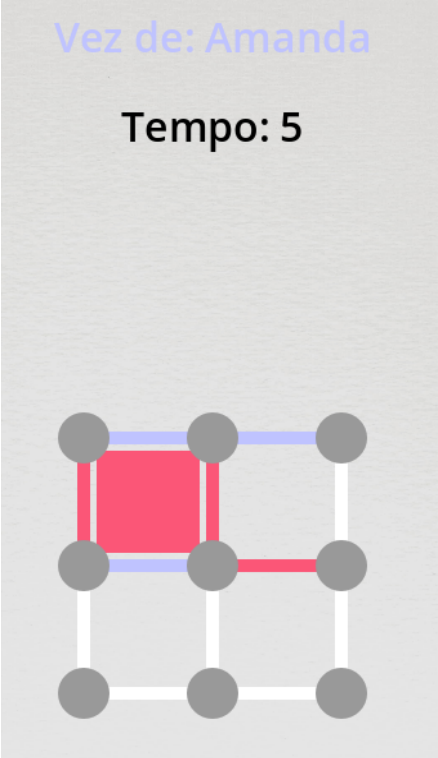
Entrar em Sala



Tabuleiro inicial



Jogada própria



Jogada do adversário

Ranking Final

Referências

FETTE, Ian; MELNIKOV, Alexey. *The WebSocket Protocol*. Fremont, CA: IETF – Internet Engineering Task Force, 2011. (RFC 6455). Disponível em: https://www.rfc-editor.org/rfc/rfc6455

VAN STEEN, Maarten; TANENBAUM, Andrew S. *Distributed Systems*. 4. ed. [S.l.]: distributed-systems.net, 2023. Disponível em: https://www.distributed-systems.net/

GODOT ENGINE. **Documentação da Godot Engine** (versão utilizada). [S.l.]: Godot Engine Community, [ano]. Disponível em: <docs.godotengine.org> Acesso em: 15 jan. 202